Manual de Utilização — RestoreDB

RestoreDB — Diagnóstico, Reparo e Manutenção de Bancos SQL Server (Módulo de Faturamento)

Ferramenta para desktop (Windows) que ajuda a **diagnosticar, reparar, restaurar e manter** bancos de dados SQL Server — em estados problemáticos (`SUSPECT`, `RECOVERY_PENDING`, `EMERGENCY`, `RECOVERING`, `RESTORING`, `OFFLINE`) **ou aparentemente saudáveis** (`ONLINE`) — com linguagem acessível a usuários não técnicos. Compatível com **SQL Server 2008 R2** e superiores.

> **Princípio central:** a ferramenta sempre tenta **primeiro o caminho SEM perda de dados**. Qualquer operação que possa descartar dados exige **confirmação explícita** — e, quando há risco de perda, é possível **simular antes** para ver exatamente o que seria descartado.

Sumário

1. Antes de começar (requisitos)
2. Conceitos importantes
3. Tela de conexão
4. Tela principal — visão geral
5. Entendendo os estados dos bancos
6. Passo a passo do reparo
7. Diagnóstico — o que é verificado
8. As opções de reparo em detalhe
9. Restauração de backup (básica e avançada)
10. Manutenção e prevenção
11. Ferramentas e cenários especiais
12. Recursos auxiliares
13. Modo escuro
14. Onde ficam os registros (logs)
15. Segurança e proteções automáticas
16. Boas práticas
17. Solução de problemas (FAQ)
18. Glossário rápido

---

1. Antes de começar (requisitos)

- **Windows** com o aplicativo RestoreDB instalado.
- **Acesso a uma instância do SQL Server** (2008 R2 ou superior).
- Um **login com permissão de** `sysadmin` na instância. As operações de reparo, restauração e manutenção (`DBCC CHECKDB`, `ALTER DATABASE`, `RESTORE`, `ALTER INDEX`) exigem esse nível.
- Para a **reconstrução de log**, o **anexar arquivos** e a **concessão de permissão aos arquivos do banco** (8.7), execute a ferramenta **como Administrador** do Windows e diretamente no servidor.

> ✅ **Versões validadas:** todas as funções foram testadas no **SQL Server 2008 R2 SP3 Express** (a mesma família usada em produção) e no **SQL Server 2025**, com autenticação SQL e do Windows.

> A ferramenta conecta-se sempre ao catálogo `master` — nunca diretamente ao banco com problema. Isso é o procedimento correto e seguro.

---

2. Conceitos importantes

- **Diagnóstico primeiro, ação depois.** Para qualquer banco — inclusive um que parece saudável — a ação inicial recomendada é **diagnosticar** (somente leitura). Só depois a ferramenta libera **apenas as opções que fazem sentido** para a situação encontrada.
- `ONLINE` **não é garantia de saúde.** Um banco pode estar `ONLINE` e, ainda assim, ter corrupção física ou de integridade (ex.: um registro apontando para outro que não existe mais). Por isso você pode **diagnosticar qualquer banco**.
- **Sem perda de dados sempre que possível.** A ferramenta tenta religar/recuperar, reconstruir índices ou corrigir a integridade antes de qualquer caminho destrutivo.
- **Simular antes de arriscar.** Quando o único caminho é o reparo com perda de dados, você pode **simular numa cópia** e ver a **perda exata por tabela** antes de decidir.
- **Confirmação explícita** para tudo que pode perder dados (caixa de seleção + diálogo).
- **Tudo é registrado** no painel **Registro de operações** e em arquivo (veja a seção 14).

---

3. Tela de conexão

Ao abrir a ferramenta, aparece a tela **RestoreDB — Conexão**.

Campo	O que informar
SERVIDOR	Nome ou endereço da instância. Ex.: `localhost`, `.\SQLEXPRESS`, `192.168.0.10,1433`
Usar autenticação do Windows (Integrated Security)	Marque para entrar com o usuário do Windows. Ao marcar, os campos Usuário/Senha são desativados.
USUÁRIO / **SENHA**	Usados apenas na autenticação do SQL Server (quando a caixa acima está **desmarcada**). Ex.: usuário `sa`.

Clique em **Conectar**.

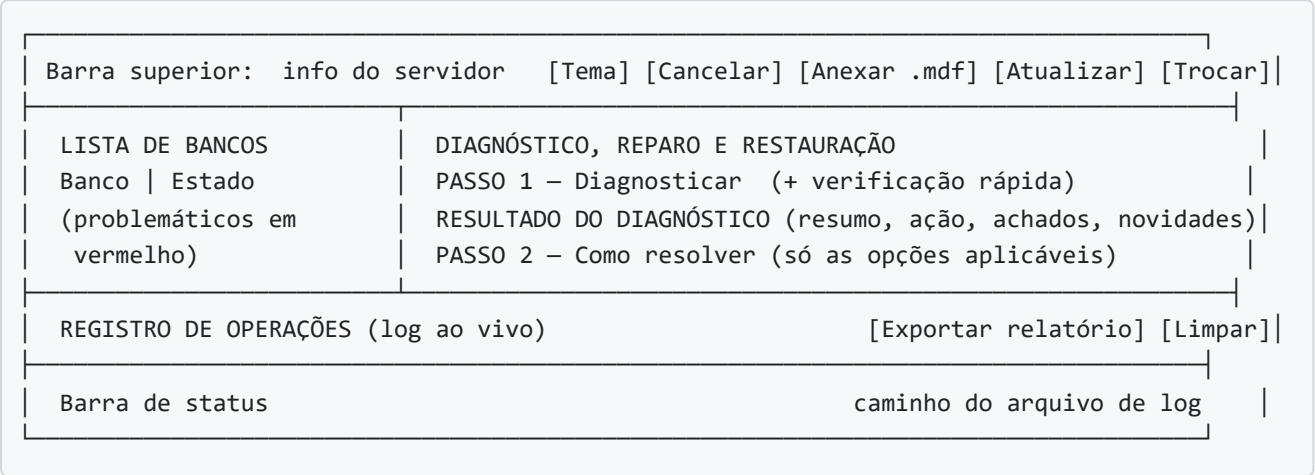
Pré-checagem automática: logo após conectar, a ferramenta **avisa** caso o login **não seja** `sysadmin` (os reparos provavelmente falharão) ou a instância seja **Azure SQL Database** (onde os comandos de reparo locais não se aplicam). Você pode continuar mesmo assim, mas o ideal é um login `sysadmin`.

> Os demais parâmetros (tempo limite, criptografia, *application name*, etc.) são pré-definidos e compatíveis com o TLS antigo do **SQL Server 2008 R2** (um aviso benigno de "TLS 1.0" pode aparecer).

---

4. Tela principal — visão geral

Após conectar, abre a tela principal, dividida em quatro áreas. A **lista de bancos** é compacta à esquerda e o **painel de ações** ocupa o maior espaço à direita (você pode arrastar o divisor entre eles).



- **Barra superior:** `Servidor`, `Catálogo: master`, edição/versão e se o login é `sysadmin`. À direita: **Tema**, **Cancelar**, **Anexar .mdf** (veja 11.4), **Atualizar lista** e **Trocar conexão**.
- **Lista de bancos:** todas as bases e seu **Estado**; problemáticos **em vermelho**.
- **Painel de ações:** segue o fluxo **Passo 1** → **Passo 2**. Inclui dois grupos recolhíveis: **"Verificações avançadas"** e **"Manutenção e ferramentas"**.
- **Registro de operações:** cada passo, colorido por tipo (informação, comando, sucesso, aviso, erro).
- **Barra de status:** mensagem atual e o caminho do arquivo de log do dia.

---

5. Entendendo os estados dos bancos

Estado	Situação	É problema?	Caminho típico
ONLINE	Disponível. Pode estar saudável **ou** corrompido sem aparentar.	Talvez	**Diagnosticar** para confirmar
OFFLINE	Desligado manualmente. Religável sem perda.	Sim	Religar (modo seguro)
RESTORING	Em restauração / aguardando finalização.	Sim	Finalizar recuperação (sem perda)
RECOVERING	Em recuperação automática.	Sim	Aguardar / finalizar (sem perda)
RECOVERY_PENDING	Recuperação pendente — geralmente falta de arquivo/recurso.	Sim	**Diagnosticar**: arquivo inacessível ou log cheio têm correção própria, sem perda
SUSPECT	Suspeito — possível corrupção.	Sim	Diagnóstico + reparo (1º sem perda)
EMERGENCY	Modo de emergência (somente leitura, usado durante o reparo).	Sim	Diagnóstico + reparo (1º sem perda)

> Em estados demorados (`RECOVERING` / `RESTORING`), o diagnóstico mostra o **progresso** da operação (“faltam ~X min”) para você saber que **não** está travado — basta aguardar.

---

6. Passo a passo do reparo

Passo 1 — Selecione o banco

Clique no banco na lista. O painel mostra o **nome**, o **ESTADO ATUAL** e a **AÇÃO RECOMENDADA**. Para **banco de sistema** (`master` , `model` , `msdb` , `tempdb`) com problema, o reparo automático **não é permitido** — há um botão **“Ver procedimento manual recomendado”** (veja 11.5).

Passo 2 — Diagnostique (não altera o banco)

Clique em **“🔍 Diagnosticar (sem alterar o banco)”**. É **somente leitura**.

- Para bancos **muito grandes**, marque **“Verificação rápida (PHYSICAL_ONLY)”** para uma checagem física mais veloz.
- O que é verificado está na seção 7.

Passo 3 — Leia o resultado

Aparece o painel **RESULTADO DO DIAGNÓSTICO**, com:

- um **resumo** e a linha **“→ Ação recomendada: …”** (cor indica a gravidade);
- ⚠ **“NOVO desde a última verificação”**, quando algo piorou em relação a uma checagem anterior;
- **POLÍTICA DE BACKUP / SAÚDE LÓGICA**: avisos de RPO (ex.: FULL sem backup de log), índices desabilitados, dependências quebradas e **causa-raiz** (erros de E/S, corrupção recorrente).

Passo 4 — Execute a opção recomendada

Em **“PASSO 2 — COMO RESOLVER”**, a ferramenta exibe **apenas a(s) opção(ões) aplicável(is)**.

O diagnóstico concluiu…	O que aparece em "Como resolver"	Perde dados?
Nenhuma corrupção	Nada a fazer (ou **Reparar — modo seguro**, se o banco estiver fora do ar)	Não
Corrupção só em índices	**Reparar índices** (8.1)	Não
Arquivo ausente / sem permissão / preso	**Corrigir acesso aos arquivos e religar** (8.7)	Não
Log de transações cheio (9002)	**Resolver log cheio** (8.6)	Não
Registros órfãos (chave estrangeira)	**Corrigir integridade** — pai recuperado do backup; o resto vai para a quarentena (8.3)	Não*
Corrupção de dados	**Restaurar de backup** / **restaurar páginas**; alternativa: reparo seguro → reparo com perda (8.4)	Só no reparo com perda (e as linhas voltam do backup quando a cadeia permite)
Problema no log (`.ldf`)	Reparo seguro e **Reconstruir o log** (8.5)	Talvez

* As linhas saem da tabela, mas ficam guardadas na quarentena (`RestoreDB_Quarentena`).

> A qualquer momento, **"Ver script planejado (não executa nada)"** mostra o T-SQL que *seria* executado (para um DBA revisar ou rodar manualmente).

---

7. Diagnóstico — o que é verificado

O diagnóstico cruza **fontes oficiais** e é **somente leitura**:

- **Consistência física** — `DBCC CHECKDB` (estruturado): identifica objeto/índice afetado e o nível mínimo de reparo. A verificação completa inclui **valores inválidos** (`DATA_PURITY`) e, se o disco estiver cheio demais para o snapshot interno, repete automaticamente de outro modo (`TABLOCK`) em vez de falhar. Opção `PHYSICAL_ONLY` para uma varredura rápida.
- **Integridade lógica** — `DBCC CHECKCONSTRAINTS`: encontra **registros órfãos** (chave estrangeira apontando para um registro inexistente) — o que o CHECKDB **não** detecta. Quando há corrupção **física**, esta checagem é **adiada** (ela lê os dados e poderia interromper a sessão) — repare o físico primeiro e diagnostique de novo.
- **Constraints desabilitadas** (desligadas de propósito pelo sistema, `NOCHECK`) são apenas **informadas** — as linhas que as violam **não** são tratadas como corrupção nem removidas.
- **Metadados** — `DBCC CHECKCATALOG`.
- **Páginas suspeitas** (`msdb.suspect_pages`) e **log de erros** do SQL (erros 823/824/825 = E/S, 9002 = log cheio), reconciliados com o estado atual (sem falso positivo de corrupção já resolvida).
- **Saúde lógica adicional** — índices **desabilitados**, **dependências quebradas**, constraints **não confiáveis**.
- **Política de backup (RPO)** — modelo de recuperação × histórico de backups (alerta FULL sem log, full antiga, etc.), classificado por severidade.
- **Causa-raiz** — erros de E/S e **corrupção recorrente** (via histórico) ⇒ suspeita de hardware/volume.

- **Arquivos do banco** — confere **no servidor** se cada `.mdf` / `.ndf` / `.ldf` existe e lê no log de erros as falhas de abertura (**arquivo ausente**, **sem permissão** — erro 5 —, **preso por outro processo** — erro 32). Isso **não é corrupção** e tem ação própria (8.7).
- **Log de transações** — % usado, o que impede a reutilização (`LOG_BACKUP`, transação aberta, replicação...) e se os arquivos de log ainda podem crescer. Log cheio (9002) **não é corrupção** e tem ação própria (8.6).
- **Topologia** (espelhamento/AG/log shipping/replicação), **espaço em disco**, `PAGE_VERIFY`, **criptografia (TDE)** e **progresso** de recuperações em andamento.
- **Histórico** — cada diagnóstico é guardado e comparado com o anterior para apontar o que é **novo**.

---

8. As opções de reparo em detalhe

A opção exibida depende do diagnóstico:

8.1. Reparar índices (sem perda de dados)

Quando: a corrupção está **restrita a índices não-clusterizados** (redundantes). **O que faz:** reconstrói (`ALTER INDEX … REBUILD`) os índices afetados e verifica. **Resultado:** banco consistente, **zero perda**.

8.2. Reparar (modo seguro, sem perda de dados)

Quando: `OFFLINE`, `RESTORING`, `RECOVERING`, `RECOVERY_PENDING` e casos em que o caminho seguro basta; também para um banco `ONLINE` **corrompido** (repara em acesso exclusivo, sem precisar de emergência). **O que faz (escalonamento automático):** religar / finalizar recuperação / `REPAIR_REBUILD`, com verificação. Se **não** bastar, a ferramenta passa a oferecer o reparo com perda — que **exige confirmação** (8.4).

8.3. Corrigir integridade (remover registros órfãos)

Quando: o diagnóstico encontrou **violações de chave estrangeira** (registros órfãos), com os arquivos físicos íntegros. **O que faz:** obtém acesso exclusivo, faz **backup de segurança** e, **antes de remover qualquer linha, tenta recuperar o "registro-pai" que faltava a partir de um backup** — assim o registro **filho** (que normalmente está saudável, apenas sem referência) é **preservado**. Só as linhas cujo pai **não existir em backup nenhum** são **removidas** (só violações de chave estrangeira; violações de CHECK são apenas reportadas). Reverifica ao final.

As linhas "removidas" não se perdem — vão para a QUARENTENA. Na mesma transação em que saem da tabela, elas são copiadas para o banco `RestoreDB_Quarentena` (na mesma instância), numa tabela `<banco>__<schema>__<tabela>` com a **data** e a **constraint** de origem. Quando o registro-pai for recadastrado, dá para devolvê-las (`INSERT … SELECT` a partir da quarentena). Se a quarentena não puder ser usada, a exclusão simples é feita (há backup de segurança antes).

> ℹ️ Gatilhos (triggers) da tabela **não disparam** nessas correções: é um reparo de dados, não uma operação de negócio — um gatilho poderia, por exemplo, estornar estoque.

Recuperação do registro-pai a partir de backup (opcional). No painel há o bloco **"RECUPERAÇÃO DE REGISTROS-PAI"**:

- **Automático** (padrão): usa o **backup FULL mais recente** do histórico do servidor.
- 📂 **Escolher .bak…**: aponte um arquivo específico — útil quando o backup está em **outra mídia** (HD externo, pasta de rede) que o histórico do servidor nem conhece. O botão ↺ **Automático** volta ao padrão.
- Se a recuperação **falhar ou recuperar só parte** (arquivo ilegível, sem um backup FULL, ou sem o pai procurado), a ferramenta **pergunta se você quer escolher OUTRO** `.bak` e tentar de novo, mirando só os órfãos restantes — e **só remove os órfãos** quando você optar por **prosseguir**. Nada é excluído sem essa decisão.

Atenção: a remoção é destrutiva (descarta linhas) — exige confirmação. Para preservar 100% dos registros, prefira **restaurar de backup**.

> 📂 O arquivo `.bak` escolhido precisa ser **legível pelo SQL Server** (a restauração roda no servidor, não no aplicativo). Para servidor remoto, informe um caminho de **rede (UNC)** acessível à conta de serviço do SQL Server.

8.4. Reparo COM possível perda de dados (exige confirmação)

Quando: o reparo seguro não resolveu e não há (ou não se quer) restaurar de backup. O painel de **restauração de backup** aparece lado a lado — é a alternativa que preserva 100% dos dados. **Simule antes:** clique em **"🔬 Simular antes (pré-visualizar a perda, sem tocar no banco)"** — a ferramenta restaura uma **cópia descartável**, executa o reparo **na cópia** e mostra a **perda exata por tabela** e se o banco ficaria limpo. **O banco real não é tocado. O que faz (ao confirmar):** primeiro a **rede de segurança automática** — captura as transações mais recentes (**tail-log**, que funciona mesmo com os arquivos de dados danificados) e um backup de segurança (`COPY_ONLY`); depois executa `DBCC CHECKDB … REPAIR_ALLOW_DATA_LOSS`, que **descarta** páginas corrompidas. Em seguida, **reconcilia** as chaves estrangeiras que a operação possa ter deixado órfãs — **recuperando o registro-pai a partir de um backup quando possível** (mesmo mecanismo da seção 8.3, inclusive a opção de escolher um `.bak` em outra mídia) e removendo só os órfãos irrecuperáveis —, **reajusta as identidades** e cria um **backup FULL novo** (os backups antigos deixam de ser confiáveis após o reparo). Os arquivos ficam na **pasta padrão de backups do servidor**. **Devolução automática das linhas descartadas (padrão: ligada).** Com a opção **"Devolver automaticamente, a partir dos backups, as linhas que o reparo descartar"**, depois do reparo a ferramenta compara a contagem de linhas **antes × depois** (e as páginas que o próprio reparo diz ter descartado), restaura numa **cópia temporária** a cadeia de backups **até o instante do reparo** (backup FULL → diferencial → logs → o tail-log recém-capturado) e **devolve só as linhas que faltam** (por chave primária, sem duplicar, com gatilhos desligados). A cópia é descartada ao final. Só é automático quando a cópia retrata **exatamente** o momento anterior ao reparo — tail-log capturado e cadeia contínua (modelo FULL). Sem isso (modelo SIMPLE, log ilegível, cadeia quebrada), a ferramenta **informa a perda por tabela** e orienta a revisão manual (**Recuperar para cópia** + **Assistente de salvamento**) — um backup mais antigo traria de volta também registros apagados de propósito. **Como usar:** marque **"Estou ciente e autorizo…"**, clique em **"Executar reparo COM perda de dados"** e confirme.

> ⚠️ **Pode haver perda de dados** — prefira sempre **restaurar de backup**. Mas, se o banco tem **cadeia de backups** (modelo FULL), o tail-log capturado antes do reparo permite depois **restaurar até o instante imediatamente anterior** e recuperar 100% do que foi descartado (peça ajuda ao suporte de nível 2/DBA usando a restauração ponto-no-tempo).

8.5. Reconstruir o log (último recurso)

Quando: o problema está no **log de transações** (`.ldf` corrompido/ausente) e o reparo não resolveu. A opção só aparece quando o **estado do banco indica** problema de log/recuperação (`SUSPECT`, `RECOVERY_PENDING`, `EMERGENCY`) ou quando o diagnóstico a recomendou — nunca como "saída" genérica para outras falhas. **O que faz:** coloca em emergência, **desanexa**, **remove** o `.ldf` antigo (preservando-o como `.corrupt_AAAAMMDDHHMMSS`) e o **recria** (`FOR ATTACH_REBUILD_LOG`), forçando o retorno a `ONLINE`. Ao final, cria um **backup FULL novo** (a reconstrução quebra a cadeia de backups de log). **Requisitos:** arquivos **acessíveis nesta máquina** e ferramenta **como Administrador**.

> 🔒 Sem Administrador, a operação é **abortada ANTES de desanexar** — o banco **não é alterado** — e as instruções manuais são exibidas. Alternativa sem Administrador: o **reparo com perda de dados** (8.4) também reconstrói o log, inteiramente no servidor.

8.6. Resolver log cheio (sem perda de dados)

Quando: o diagnóstico acusa **log de transações cheio** (erro 9002) ou prestes a encher. Não é corrupção — o banco recusa gravações (ou não sobe) porque o `.ldf` não tem onde crescer. **O que faz (do menos ao mais invasivo):**

1. descobre **o que segura o log**: se falta **backup de log** (modelo FULL), faz o `BACKUP LOG` na pasta padrão de backups (**guarde o arquivo** — ele faz parte da cadeia); se é uma **transação aberta**, mostra **quem** (sessão, máquina, programa, desde quando) — a ferramenta **não encerra sessões por conta própria**; replicação/espelho atrasado → orientação;
2. se o arquivo de log chegou ao **limite** ou está **sem crescimento automático** e há espaço no disco, libera o crescimento;
3. se o **disco do log está cheio**, cria um **arquivo de log extra** no disco da instância com mais espaço livre (depois pode ser removido — o comando aparece no registro);
4. se o banco estava fora do ar (`RECOVERY_PENDING` / `SUSPECT` por log cheio), **religa**.

8.7. Corrigir acesso aos arquivos e religar (sem perda de dados)

Quando: o SQL Server **não consegue abrir** um arquivo do banco — `RECOVERY_PENDING` / `SUSPECT` depois de trocar disco/letra de unidade, mexer em permissões, antivírus bloqueando etc. **Não é corrupção**, e nenhum reparo de DBCC resolveria. **O que faz:**

- **Arquivo ausente** (foi movido): clique em **"📂 Localizar arquivo ausente..."** e aponte onde ele está agora; a ferramenta confere que o **servidor** enxerga o caminho e atualiza o catálogo (`ALTER DATABASE … MODIFY FILE`).
- **Sem permissão** (erro 5): concede **Controle Total à conta de serviço do SQL Server** nos arquivos (exige a ferramenta **como Administrador**, no servidor; sem isso, os comandos `icacls` para o suporte aparecem no registro).
- **Arquivo preso por outro processo** (erro 32): orienta a fechar antivírus/backup de arquivos/cópias.
- Por fim, **religa** o banco (`SET ONLINE`) — a recuperação normal do SQL Server roda e **nada é perdido**. Rode o diagnóstico de novo para confirmar.

---

9. Restauração de backup (básica e avançada)

Quando o diagnóstico aponta **corrupção de dados** (`.mdf` / `.ndf`), **restaurar é o caminho correto**.

9.1. Restauração básica

1. Selecione um backup do **histórico** (`msdb`) **ou** clique em **"Escolher arquivo .bak..."**.
2. Deixe **"Substituir o banco existente (REPLACE)"** marcado.
3. Clique em **"Restaurar o backup selecionado"** e confirme. A ferramenta verifica a mídia antes e **corrige usuários órfãos** ao final.

9.2. Restaurar cadeia completa (mais recente)

"Restaurar cadeia completa (mais recente)" monta automaticamente **FULL → diferencial → logs** a partir do histórico, **captura o tail-log** (transações mais recentes) e valida a **continuidade** antes de executar — se a cadeia estiver quebrada, **aborta** sem alterar o banco.

9.3. Mais opções de restauração (avançado)

Recolhidas sob **"Mais opções de restauração (avançado)"**:

- **Restaurar até um ponto no tempo (STOPAT):** marque a opção, escolha **data e hora** e use a cadeia/STANDBY para restaurar **até aquele instante**.
- **Testar backup (restauração de teste):** restaura num banco **descartável** e roda `DBCC CHECKDB`, comprovando que o backup é **íntegro e restaurável** — **sem tocar na produção**.
- **Verificar saúde dos backups:** roda `RESTORE VERIFYONLY` em todo o histórico.
- **Recuperar para cópia (salvamento):** restaura num banco novo (`..._Recuperado`) para **extrair dados** sem tocar no original. Em seguida, o **Assistente de salvamento** copia de volta ao banco vivo apenas as **linhas que faltam** (por chave primária, sem duplicar).
- **Restaurar em STANDBY (somente leitura):** deixa o banco **consultável** entre os logs, para você inspecionar e achar o ponto certo; depois use **"Finalizar recuperação"**.
- **Restaurar apenas as páginas afetadas:** quando poucas páginas estão corrompidas, recupere **só elas** (exige modelo FULL/BULK e a cadeia de backups). Quando o diagnóstico recomenda esse caminho, o campo já vem **preenchido automaticamente** com as páginas corrompidas — normalmente basta clicar em **"Restaurar apenas estas páginas"**. A cadeia (FULL → diferencial → logs + um backup de log novo) é montada e validada automaticamente; se estiver incompleta, a operação é abortada com explicação, sem alterar o banco.

> Backups **divididos em vários arquivos (striped)** e **comprimidos** são reconhecidos e tratados automaticamente.

---

10. Manutenção e prevenção

No grupo recolhível **"Manutenção e ferramentas"**:

10.1. Otimizar índices e estatísticas (manutenção preventiva)

"Otimizar índices e estatísticas" desfragmenta os índices conforme a fragmentação (10–30% ⇒ `REORGANIZE`; acima de 30% ⇒ `REBUILD`) e atualiza as estatísticas. **Não é destrutivo.** Use periodicamente para manter o desempenho.

10.2. Verificação agendada (monitoramento + alerta)

"Configurar verificação agendada (gerar comando)" mostra um comando pronto do **Agendador de Tarefas do Windows** que roda uma verificação diária (via utilitário `RestoreDB.Probe`). Quando aparece **algo novo** (corrupção/violações), é gravado um **alerta** em `%LOCALAPPDATA%\RestoreDB\Alerts` e o utilitário sinaliza com um **código de saída** que o Agendador pode usar para **enviar e-mail** ao suporte. É a **detecção precoce** de problemas.

> O utilitário usa a **autenticação do Windows** da conta que executa a tarefa (recomendado). Se a instância só aceitar **autenticação SQL**, defina também as variáveis `PROBE_USER` e `PROBE_PASS` na tarefa — lembre-se de que a senha fica gravada na configuração da tarefa; use um login dedicado.

---

11. Ferramentas e cenários especiais

11.1. Recuperar de exclusão acidental (forense)

"Procurar exclusões recentes no log" lê o log de transações e lista **DELETE/DROP/TRUNCATE** recentes com horário. Cada item mostra também a **tabela** e a quantidade de linhas. Selecione a transação e escolha:

- **"Devolver só as linhas excluídas (não sobrescreve o banco)"** — *recomendado para DELETE*: a ferramenta restaura uma **cópia temporária** até **logo antes** do DELETE e devolve à tabela **só as linhas que faltam** (por chave primária, sem duplicar, gatilhos desligados). **Tudo o que foi feito depois continua no banco.** (Linhas dessa tabela apagadas de propósito depois do instante escolhido também voltariam — a confirmação avisa.)
- **"Restaurar o banco inteiro até ANTES da transação"** — para DROP/TRUNCATE: substitui o banco pela cadeia de backups até logo antes do evento; **o que veio depois é descartado**.

Ambas exigem modelo **FULL** e a cadeia de backups disponível. *Observação:* o log mostra apenas a parte **ativa** — quanto antes, melhor a chance.

11.2. Simular um reparo destrutivo

Antes de qualquer reparo com perda, use **"🔬 Simular antes"** (ver 8.4): você vê a **perda exata por tabela** numa cópia, sem risco para a produção.

11.3. Verificações avançadas

No grupo **"Verificações avançadas"**:

- **Verificar tabela (CHECKTABLE):** checagem dirigida a uma tabela (mais rápida que o CHECKDB completo).

- **Reajustar identidades (CHECKIDENT RESEED):** corrige as sementes de colunas de identidade após remoção de linhas (reparo com perda / salvamento).

11.4. Anexar um banco a partir de um arquivo `.mdf`

Na barra superior, **"Anexar .mdf"** permite recuperar um banco a partir do arquivo de dados solto. A ferramenta procura o `.ldf` correspondente pelos padrões comuns; se não encontrar, **pergunta se você quer localizá-lo manualmente** — só recria o log (`FOR ATTACH_REBUILD_LOG`) se você optar por isso ou se o arquivo realmente não existir (recriar descarta transações não confirmadas). Não sobrescreve um banco já existente de mesmo nome.

11.5. Procedimento manual para bancos de sistema

Bancos de sistema não são reparados automaticamente. **"Ver procedimento manual recomendado"** exibe o **runbook** específico (`master` / `model` / `msdb` / `tempdb`) com os passos a executar no servidor (modo de usuário único, restauração do banco de sistema, etc.). Acione o DBA quando possível.

---

12. Recursos auxiliares

- **Atualizar lista / Trocar conexão / Cancelar:** como antes.
- **Ver script planejado (não executa nada):** mostra o T-SQL que seria executado.
- **Exportar relatório:** salva o **Registro de operações** em **HTML** (anexe a um chamado).
- **Limpar:** limpa o registro exibido (não apaga o arquivo de log).

---

13. Modo escuro

Ao **iniciar**, a ferramenta adota o **tema do Windows** (claro/escuro). Você pode alternar pelo botão **"🌙 Tema escuro" / "☀ Tema claro"** na barra superior; a escolha vale para toda a sessão.

---

14. Onde ficam os registros (logs)

```
%LOCALAPPDATA%\RestoreDB\Logs\repair-AAAAMMDD.log     (um arquivo por dia)
%LOCALAPPDATA%\RestoreDB\History\<banco>.json         (histórico de diagnósticos)
%LOCALAPPDATA%\RestoreDB\Alerts\                       (alertas da verificação agendada)
```

O caminho do log do dia também aparece na **barra de status**. Para entregar o histórico de uma sessão a outra pessoa, use **Exportar relatório** (HTML).

---

15. Segurança e proteções automáticas

- **Diagnóstico não altera nada** (somente leitura).
- **Simulação na cópia:** o reparo destrutivo pode ser pré-visualizado **sem tocar na produção**.

- **Guarda de topologia:** espelhamento/Always On/log shipping/replicação **bloqueiam** o reparo destrutivo, com orientação para tratar a topologia primeiro.
- **Bancos de sistema** não sofrem reparo destrutivo automático (há runbook guiado).
- **Sempre via `master`:** nunca se conecta diretamente ao banco quebrado.
- **Rede de segurança automática antes do reparo destrutivo:** captura das transações mais recentes (**tail-log**, mesmo com os dados danificados) + backup de segurança + cópia dos arquivos (quando possível). Com cadeia de backups, o que o reparo descartar **pode ser recuperado depois**.
- **Recuperação do registro-pai antes de remover órfãos:** ao corrigir integridade (ou reconciliar chaves estrangeiras após um reparo com perda), a ferramenta tenta **trazer o registro-pai de volta a partir de um backup** — preservando os filhos saudáveis — e só remove o que não existir em backup nenhum. Pode-se **apontar um `.bak` de outra mídia** e, se a recuperação ficar incompleta, ela **pergunta antes de excluir**.
- **Backup FULL novo após reparo destrutivo/reconstrução de log** — nova base da cadeia de backups.
- **Linhas descartadas pelo reparo com perda são devolvidas dos backups** quando a cadeia chega até o instante do reparo; **órfãos removidos vão para a quarentena** (`RestoreDB_Quarentena`), não para o lixo.
- **Gatilhos não disparam** ao devolver linhas (salvamento, registro-pai, reparo) nem ao mover órfãos.
- **Constraints desabilitadas pelo sistema são respeitadas** — nunca geram remoção de linhas.
- **Reconstruir índices não desliga chaves estrangeiras:** as FKs que o SQL Server desabilita junto com um índice único/PK são reativadas ao final.
- **Restaurar com a aplicação conectada** funciona: as conexões são encerradas no mesmo comando do `RESTORE`, e o banco volta em acesso normal (`MULTI_USER`).
- **Backups cujo arquivo foi apagado** (limpeza de retenção) e **cópias de segurança da própria ferramenta** (que retratam o banco já danificado) **não** são escolhidos como base de restauração.
- **Cadeias de restauração validadas** antes de executar; conjuntos que não podem ser restaurados sozinhos (diferencial/log) geram **aviso com orientação**.
- **Robustez a falhas graves:** se o SQL Server derrubar a conexão ao ler uma página corrompida (erro 824), a ferramenta **reconecta sozinha** e continua; o acesso exclusivo é de fato mantido pela sessão durante o reparo.
- **Fechar durante uma operação pede confirmação** — evita abandonar o banco em acesso restrito.
- **Confirmação explícita** para tudo que pode perder dados; **sem mensagens técnicas cruas**.

---

16. Boas práticas

1. **Tenha backups** — e verifique a **saúde** deles (9.3). É a única forma de recuperar 100% dos dados.
2. **Configure a verificação agendada** (10.2) — detecte a corrupção cedo.
3. **Diagnostique antes de agir**, inclusive em bancos `ONLINE`.
4. **Simule** antes de qualquer reparo com perda de dados.
5. **Prefira restaurar de backup** a descartar dados.
6. Em **alta disponibilidade**, trate a topologia antes de qualquer reparo destrutivo.
7. **Manutenção preventiva** periódica (índices + estatísticas) mantém o desempenho.
8. Para reconstrução de log / anexar arquivos, rode **como Administrador**, no servidor.

---

17. Solução de problemas (FAQ)

Não consigo conectar / "Login failed". Verifique servidor e credenciais; para Windows, marque a caixa correspondente.

Conectou, mas avisa que "não é sysadmin". Os reparos exigem `sysadmin`. Conecte-se com um login adequado.

As opções de reparo não aparecem. Elas só são liberadas **após o diagnóstico**. Selecione o banco e clique em **"Diagnosticar"**.

O banco está ONLINE — preciso diagnosticar? Sim, se desconfiar de problemas: `ONLINE` não garante ausência de corrupção. O diagnóstico é seguro (somente leitura).

Quero ver quanto perderia antes de reparar. Use **"🔬 Simular antes"** no painel de reparo com perda — ele mostra a perda por tabela numa cópia, sem tocar na produção.

Apaguei dados sem querer. Use **"Procurar exclusões recentes no log"** (11.1) e, para um DELETE, **"Devolver só as linhas excluídas"** — o resto do banco não é tocado. Exige modelo FULL e cadeia de backups.

O banco está RECOVERY_PENDING depois de trocar o disco / a letra da unidade. Diagnostique: se aparecer **arquivo ausente** ou **sem permissão**, use **"Corrigir acesso aos arquivos e religar"** (8.7) — aponte o novo local do arquivo; nada é perdido.

Aparece "log de transações cheio" (erro 9002). Não é corrupção. Use **"Resolver log cheio"** (8.6). Se o motivo for uma transação aberta, a ferramenta mostra qual é — conclua-a ou encerre-a com o responsável.

Onde estão as linhas órfãs que a correção de integridade removeu? No banco `RestoreDB_Quarentena`, tabela `<banco>__<schema>__<tabela>`, com a data e a constraint de origem. Podem ser devolvidas quando o registro-pai for recadastrado.

Aparece um aviso de topologia (espelhamento/AG/replicação). O reparo destrutivo foi bloqueado por segurança. Trate a topologia primeiro e tente novamente.

A reconstrução de log / anexar diz que exige Administrador. Reabra a ferramenta **como Administrador**, no servidor. Para o log, a alternativa é o **reparo com perda de dados**, que não mexe em arquivos.

O diagnóstico mostra páginas suspeitas antigas, mas o CHECKDB está limpo. A ferramenta reconcilia histórico × estado atual: corrupção **já resolvida** não gera falso positivo.

Aparece "Corrupção RECORRENTE" / erros de E/S. É um alerta de **causa-raiz**: suspeite do disco/volume/storage e considere migrar os arquivos.

Autorizei o reparo com perda de dados — dá para recuperar o que foi descartado? Se o banco usa o modelo de recuperação **FULL** e tem cadeia de backups, **sim**: antes do reparo a ferramenta capturou o **tail-log** (arquivo `*_tail_*.trn` na pasta padrão de backups do servidor). Uma **restauração ponto-no-tempo** até o instante imediatamente anterior ao reparo recupera 100% dos dados. Peça apoio ao nível 2/DBA e informe o horário do reparo (está no Registro de operações).

Posso fechar a ferramenta durante um reparo? Evite. A ferramenta pede confirmação: fechar no meio pode deixar o banco em acesso restrito (`SINGLE_USER`). Prefira **Cancelar** e aguardar.

Selecionei um backup e a restauração diz que não é FULL. Diferenciais e logs não se restauram sozinhos — precisam do FULL antes. Use **"Restaurar cadeia completa"**, que monta a sequência correta automaticamente.

Depois do reparo com perda, os backups antigos ainda valem? Não confie neles: o reparo altera a estrutura do banco. A ferramenta cria automaticamente um **backup FULL novo** (arquivo `*_PosReparo_*.bak`) — ele é a nova base. Configure a rotina de backups a partir dele.

Ao corrigir integridade, dá para preservar os registros filhos em vez de apagá-los? Sim. Antes de remover um órfão, a ferramenta tenta **recuperar o registro-pai** que faltava a partir de um backup (o FULL mais recente do histórico) — com isso o filho é **preservado**. Se o seu backup estiver em **outra mídia** (HD externo, pasta de rede), use **"RECUPERAÇÃO DE REGISTROS-PAI → 📂 Escolher .bak..."** no painel. Se a recuperação não resolver tudo, a ferramenta **pergunta antes de remover** se você quer apontar outro `.bak`. O arquivo precisa ser **legível pelo SQL Server** (a restauração roda no servidor; para servidor remoto, use um caminho de rede UNC).

---

18. Glossário rápido

- `DBCC CHECKDB` — verifica a consistência **física** de um banco.
- `DBCC CHECKCONSTRAINTS` — verifica a integridade **lógica** (chaves estrangeiras / CHECK).
- `REPAIR_REBUILD` — reparo **sem** perda (estruturas reconstruíveis).
- `REPAIR_ALLOW_DATA_LOSS` — reparo que pode **descartar** dados corrompidos.
- **Registro órfão** — linha cuja **chave estrangeira** aponta para um registro que não existe.
- **Registro-pai / registro-filho** — numa chave estrangeira, o **pai** é o registro referenciado (a chave primária) e o **filho** é o que aponta para ele. Um filho fica **órfão** quando o pai some — a ferramenta tenta **recuperar o pai do backup** antes de remover o filho.
- **Ponto-no-tempo (STOPAT)** — restaurar até um instante específico (logo antes de um erro).
- **STANDBY** — banco restaurado em **somente leitura**, consultável entre os logs.
- **Tail-log** — backup das transações mais recentes, capturado antes de sobrescrever.
- **Striped** — backup dividido em vários arquivos. **TDE** — banco criptografado.
- **Fragmentação** — desorganização dos índices que prejudica o desempenho (resolve com manutenção).
- `.mdf` / `.ndf` — arquivos de **dados**. `.ldf` — arquivo de **log** de transações.
- **Usuário órfão** — usuário do banco sem login no servidor (corrigido após o restore).
- **Topologia de alta disponibilidade** — espelhamento, Always On (AG), log shipping ou replicação.

> Para detalhes técnicos (arquitetura, T-SQL exato e cenários de teste), consulte o `README.md` e os scripts em `sql/`.